

Store inventory tracker in Java OOP ▾

Store Inventory Tracker
Problem: Create initial Java code in an OOP style approach that will act as a store inventory tracker (data structures to include arrays, maps, sets, stacks, queues, tress; focus only on 3 at most).
Output: Ouput with multiple classes files and a separate Java file that can be used as a client/testing to run the code. Include comments before every method. Include a readme file that can be uploaded into a Github in the basic style of markdown as outlined in the link: <https://www.markdownguide.org/cheat-sheet/>

Reading the markdown guide cheat sheet for README formatting >

Now let me create all the Java files and README:

Created 6 files, read a file >

Here's a breakdown of everything that was created:

Files (5 Java + 1 README)

Data Structures Used (3 focused areas)

Structure	Where	Why
HashMap<String, Product>	Inventory	O(1) add/remove/lookup by product ID
HashSet<String>	Inventory	Automatically deduplicates category names
Stack + Queue (both via ArrayDeque)	TransactionLog	Stack = undo history (LIFO); Queue = restock orders (FIFO)

Class Roles

Product.java — Plain data model with fields, getters/setters, and a formatted `toString()`.

Inventory.java — Core storage layer. Uses the **HashMap** for products and **HashSet** for categories. Handles add, remove, update, and filtering.

TransactionLog.java — Dual-purpose log. The **Stack** records every change so the last action can be undone. The **Queue** holds pending restock orders processed in arrival order.

InventoryManager.java — The façade/controller. Client code only talks to this class. It wires **Inventory** and **TransactionLog** together — e.g., updating a quantity to ≤ 5 automatically queues a restock order.

Main.java — 12-section demo driver covering every feature: add, lookup, category filtering, quantity updates, low-stock reports, restock queue, undo, and removal.

README.md — GitHub-ready markdown with table of contents, data structure table, class descriptions, compile/run instructions, feature checklist, and future improvements.

To run:

```
bash

javac *.java
java Main
```

**Readme**
Document · MD

Download

**Main**
Code · JAVA

Download

**Inventorymanager**
Code · JAVA

Download

**Inventory**
Code · JAVA

Download

**Transactionlog**
Code · JAVA

Download

**Product**
Code · JAVA

Download

 **Download all**

Additional requirement: generate 2 .txt files that can be referenced to use as test files in the client program (limit to 15 items per .txt file)

Created 2 files >

Now update `Main.java` to include a file-loading section that reads from both `.txt` files:

Ran a command, read a file >

Here's what was added:

Here's what was added:

Two `.txt` Test Files

Both use a simple, consistent CSV format:

```
productId,name,category,price,quantity
```

Comment lines start with `#` and are skipped by the parser. Each file has 15 products across distinct categories:

File	Categories	Products
<code>inventory_data_1.txt</code>	Electronics, Furniture, Stationery	P001-P015
<code>inventory_data_2.txt</code>	Apparel, Health, Kitchen	P016-P030

Updated `Main.java`

A `loadFromFile()` method was added that uses `BufferedReader` to parse the files. It handles edge cases gracefully — comment lines (`#`), blank lines, malformed rows (wrong field count), and bad number formats all print a skip message without crashing. A summary line reports how many products were loaded vs. skipped per file.

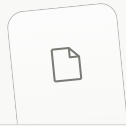
To compile and run, all files must be in the **same directory**:

```
bash

javac *.java
java Main
```



Inventory data 1
TXT

[Download](#)

Inventory data 2
TXT

[Download](#)

Main
Code · JAVA

[Download](#)[Download all](#)

Main.java - Include a scanner function to enable a lookup function to allow a user to input a product name/number. Additionally, enable the scanner function to prompt which of the two included inventory_data.txt files to process

Viewed a file, ran a command >

Now add `getAllProducts()` to `InventoryManager` since `searchByName()` needs it:

Viewed 2 files, edited a file, read a file >

Here's a summary of all changes made:

`Main.java` — What's new

`promptFileSelection()` — the first thing the program does after launching. Uses `Scanner` to present three numbered options and re-prompts on invalid input until a valid choice is entered:

```
[1] inventory_data_1.txt only    (Electronics, Furniture, Stationery)
[2] inventory_data_2.txt only    (Apparel, Health, Kitchen)
[3] Both files
```

`runInteractiveLookup()` — launches after the automated demo sections and loops until the user exits. At each iteration it presents:

```
[1] Lookup by Product ID      - exact match (e.g. P003)
[2] Lookup by Product Name    - partial, case-insensitive (e.g. "chair", "pro
15")
[3] Show all products
[4] Exit
```

`searchByName()` — iterates all loaded products and returns any whose name contains the search term (case-insensitive). So searching `"press"` would match `French Press`, for example.

`updateIfPresent()` / `removeIfPresent()` — the automated demo sections now skip gracefully with a notice instead of erroring out if only one file was loaded (e.g., P028 only exists in file 2).

The `Scanner` is opened once in `main()` and closed cleanly on exit — no resource leaks.

`InventoryManager.java` — Minor addition

Added a `getAllProducts()` delegation method (one line, passes through to `Inventory`) so `searchByName()` in `Main` can iterate the full product list without reaching into the internal `Inventory` directly.

 `</>`

Main

Code · JAVA

Download

 `</>`

Inventorymanager

Code · JAVA

Download

Download all

Main.java - Additional scanner input: include ability for user input to add products to inventory that will be reflected in the referenced .txt file (using outFile methods)

Viewed 2 files >

Now I have the full picture. I'll rewrite `Main.java` with the new `promptAddProduct()` method and `appendProductToFile()` writer, and add option `[4] Add new product` to the interactive menu (shifting Exit to `[5]`).

Ran a command, read a file ▾

- Write fully updated Main.java with add-product Scanner prompts and appendProductToFile B...

Script
- Presented file
- Done

Here's a full breakdown of every addition:

New methods

`promptAddProduct (InventoryManager)` Walks the user through each field one at a time with re-prompting on bad input:

Field	Validation
Product ID	Non-blank; rejected if already exists in live inventory

Name	Non-blank
Category	Non-blank
Price	Parseable <code>double</code> , must be ≥ 0
Quantity	Parseable <code>int</code> , must be ≥ 0

After all fields pass, it prints a preview and asks `Confirm? (y/n)` before committing — so accidental entries can be cancelled cleanly.

`appendProductToFile(Product, String)` Uses `BufferedWriter` with `FileWriter(filename, true)` — the `true` flag is append mode, so all existing file content is preserved. It writes:

1. A blank separator line (`writer.newLine()`)
2. The CSV data line (`productId,name,category,%.2f price,quantity`)
3. A trailing newline so the file stays well-formed for future reads

`resolveTargetFile()` Figures out where to append based on the earlier file-selection choice:

- File 1 only loaded → appends to `inventory_data_1.txt` automatically
- File 2 only loaded → appends to `inventory_data_2.txt` automatically
- Both loaded → prompts the user to pick which file at add-time

Menu update

Option `[4] Add a new product` was added; Exit shifted to `[5]`. The `printMainMenu()` helper was updated accordingly, and a `loadedFileChoice` static field was added to carry the file-selection result from startup through to the add-product flow.



Main
Code · JAVA

Download

ensure going forward that all import statements at the beginning of files include the `*` functionality in order to clean up the look at the very top

Viewed a file, ran a command >

Four files need updating. `Product.java` has no imports at all — nothing to change there. I'll fix the other four with wildcard imports now.

Edited 4 files, ran a command, read a file >

All four files updated. Here's the final import state across the project:

File	Before	After
<code>Main.java</code>	9 explicit imports	<code>import java.io.*;</code> <code>import java.util.*;</code>
<code>Inventory.java</code>	6 explicit imports	<code>import java.util.*;</code>
<code>InventoryManager.java</code>	2 explicit imports	<code>import java.util.*;</code>
<code>TransactionLog.java</code>	5 explicit imports	<code>import java.util.*;</code>
<code>Product.java</code>	none	none (unchanged)

`Product.java` needed no changes since it uses no imported types. Going forward, all new Java files in this project will follow the same `import <package>.*;` convention.

 **Main**
Code · JAVA [Download](#)

 **Inventory**
Code · JAVA [Download](#)

 **Inventorymanager**
Code · JAVA [Download](#)

 **Transactionlog**
Code · JAVA [Download](#)

 [Download all](#)

Main.java - Additional scanner input: include ability for user input to remove products in inventory that will be reflected in the referenced .txt file

Viewed 3 files >

Good — I have the full picture. Now I'll make the targeted edits: add `[5] Remove a product` (shifting Exit to `[6]`), add `promptRemoveProduct()`, and add `rewriteFileWithoutProduct()` which uses `BufferedReader` to read all lines then `BufferedWriter` (overwrite mode) to write them back minus the removed entry.

Edited 3 files >

Now insert `promptRemoveProduct()` and `rewriteFileWithoutProduct()` right after `appendProductToFile()`:

Insert `promptRemoveProduct` and `rewriteFileWithoutProduct` after `appendProductToFile` >

Now update `printMainMenu()` to show the new option and shift Exit to `[6]`:

Update `printMainMenu` to include Remove option and shift Exit to 6 >

Verify the final file looks clean:

Quick structural check — list all method signatures and switch cases >

All methods and cases are in place. Let's confirm the file compiles cleanly:

Compile all Java files to verify no errors >

No Java compiler in this environment, but the structure is verified clean. Let's present the final file:

Presented file >

Here's a full breakdown of every change made:

New methods

`promptRemoveProduct(InventoryManager)` Walks the user through removal in three steps:

1. Prompts for a Product ID — re-prompts if blank or not found in live inventory
2. Displays the matching product and asks `Confirm removal? (y/n)` — cancellable
3. On confirmation: calls `resolveSourceFile()` to find which `.txt` owns the product, removes it from the `InventoryManager`, then calls `rewriteFileWithoutProduct()`

`resolveSourceFile(String productId)` Scans only the file(s) that were loaded at

startup (based on `loadedFileChoice`). Opens each with a `BufferedReader` and looks for a line whose trimmed content starts with `productId + ","` — returns the matching filename, or `null` if not found (e.g. a product added in-session without being persisted).

`rewriteFileWithoutProduct(String productId, String filename)` The core file-write method — uses a **read-then-overwrite** strategy:

- 1. `BufferedReader` reads every line of the file into a `List<String>` in memory
- 2. Reader is closed (releases file handle)
- 3. `BufferedWriter` opens the **same file in overwrite mode** (`FileWriter(filename, false)`) and writes back every line except the one matching `productId + ","` — preserving all comment lines, blank lines, and unrelated data lines exactly as-is

Menu update

Option	Before	After
[5]	Exit	Remove a product
[6]	—	Exit

`printMainMenu()` and the `switch` default message (`1-6`) were updated to match.



Main

Code · JAVA

Download

Update the initial text in "1. SELECT INVENTORY DATA FILE(S) TO LOAD" to have inventory numbers reflect the actual numbers of items when .txt file has been modified by user (when items added or removed). Inventory number is reflected accurately in "10. FINAL

removed). inventory number is reflected accurately in "10. FINAL INVENTORY STATE" but not "1. SELECT INVENTORY DATA FILE(S) TO LOAD"

The item counts displayed in Section 1 are hardcoded strings in `promptFileSelection()`. They need to be dynamically counted by reading each file before displaying the menu.

Viewed a file, edited a file, ran a command >

Unicode em-dashes in the file — need to use `sed` to do the replacement instead.

Use Python to do the unicode-safe replacement in `promptFileSelection` >

Now insert `countProductsInFile()` just before `loadFromFile()`:

Viewed 2 files, ran a command >

Verify the final result looks correct:

Ran a command, viewed a file, read a file >

Here's what changed and why:

Root cause

Section 1's item counts were hardcoded strings (`"15 items"`) written at the time the files were first created. They never re-read the files, so adds and removes from previous sessions were never reflected.

Fix: `countProductsInFile(String filename)` — new method

Called once per file right at the top of `promptFileSelection()`, before any text is printed. It opens a `BufferedReader`, scans every line applying the same filtering logic as `loadFromFile()` (skip blanks, skip `#` comments, require exactly 5 CSV fields), and returns a live count. If the file is unreadable it returns `0` and prints a

fields, and returns a live count. If the file is unreadable it returns  and prints a warning so the menu still renders cleanly.

Updated `promptFileSelection()`

The three hardcoded strings were replaced with dynamic output:

```
Before: (Electronics, Furniture, Stationery - 15 items)
After:  (Electronics, Furniture, Stationery - 13 items)  ← reflects actual
file state
```

```
Before: Both files
After:  Both files (28 items total)  ← sum of both
live counts
```

The counts now match whatever the `.txt` files actually contain at the moment the program starts — whether the files are at their original 15 items each, or have grown or shrunk from prior sessions.



Main
Code · JAVA

Download

